第 3 章　銀河の中で星を動かす

第 2 章では、二つの天体を点質量として動かしました。ここからは、銀河の内部へ目を向けます。銀河の質量は中心の 1 点だけに集まっているわけではありません。中心からの距離によって、内側に含まれる質量も、星が受ける重力も変わります。本章では、その広がりを球対称の Plummer モデルで表します。

新しく学ぶ数値計算法は、**台形則**と**二分法**です。密度を積分して質量分布を求め、その質量の半分を含む半径を探します。さらに、第 1・2 章で使った KDK leapfrog で、銀河の重力場中の星を動かします。数値積分、求根、時間積分を、一つの銀河モデルの中で区別して使うことが目標です。

この章で学ぶこと

- 球対称の密度、内側の質量、ポテンシャル、重力加速度の関係を説明できます。
- 台形則を実装し、Plummer 球の質量分布を解析式と比較できます。
- 二分法で半質量半径を求め、求根の残差と半径の誤差を区別できます。
- Plummer ポテンシャルに対応する円運動速度を与え、KDK で円軌道と非円軌道を計算できます。
- 積分範囲、空間の分割、求根の許容幅、時間刻みが、それぞれ何に影響するかを確認できます。

3.1 質点のモデルを広がった銀河へ拡張する

3.1.1 星を表す 1 粒子を動かす

原点に中心を持つ、静的で球対称な銀河を考えます。その中を動く星を 1 個のテスト粒子で表します。粒子は銀河の重力を受けますが、銀河の密度やポテンシャルを変えません。銀河自身の粒子を多数用意して、その自己重力を計算するわけではありません。

第 2 章のように二つの有限質量が互いに動く問題とは違い、今回は重力場を固定します。これは銀河相互作用を始める前に、1 個の銀河の重力を理解するための段階です。銀河中心が動く場合は第 5 章で扱います。

粒子の位置を $\boldsymbol{r} = (x, y)$、速度を $\boldsymbol{v} = (v_x, v_y)$ とし、$r = |\boldsymbol{r}|$ を原点からの距離とします。銀河は 3 次元の球ですが、初期位置と初期速度を x–y 平面内に選べば、中心力による運動はその平面内にとどまります。**2 次元で軌道を描くことと、2 次元の質量分布を仮定することは別です。**[2]

3.1.2 球の大きさと計算する範囲を区別する

本章では、球全体の質量を M、その広がりを表すスケール長を a とします。a は第 2 章で使った Kepler 楕円の軌道長半径 $a_{\rm orb}$ ではありません。また、加速度ベクトル $\boldsymbol{a}$ とも区別します。

Plummer モデルの密度は、有限の半径で突然 0 にはなりません。外側ほど小さくなり、無限遠まで続きますが、全質量は有限です。後で使う $r_{\rm tab}$ は、質量の表を作る範囲の上限です。$r = r_{\rm tab}$ で密度や重力を切断するという意味ではありません。

3.2 密度から質量と重力を求める

3.2.1 薄い球殻に含まれる質量を求める

球対称なら、密度 $\rho(r)$ は半径だけの関数です。半径 r から $r + dr$ までの薄い球殻の体積は $4\pi r^2 dr$ ですから、その質量は

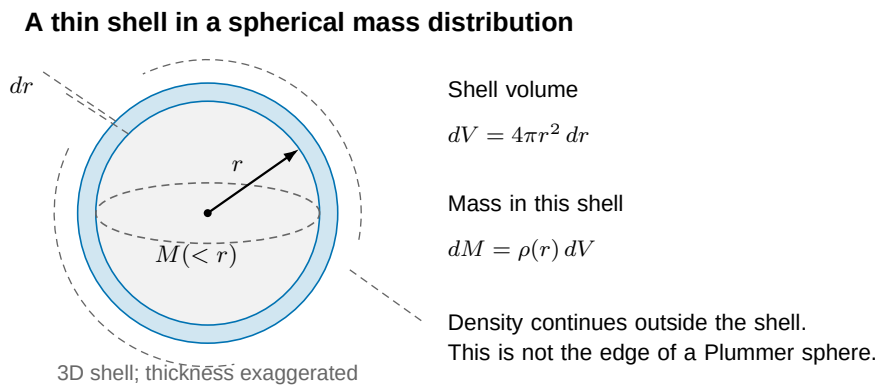
$$dM = 4\pi r^2 \rho(r)\, dr \tag{3.1}$$

です。中心から半径 r まで足し合わせると、内側の質量は

$$M(<r) = 4\pi \int_0^r r'^2 \rho(r')\, dr' \tag{3.2}$$

となります。r' は積分の途中で使う半径です。$M(<r)$ は r に依存する量、M は球全体の一定の質量として使い分けます。

図3.1では、薄い球殻とその内側を分けて示しています。図は断面ですが、積分する質量には 3 次元の球殻の体積を使います。



$$M(< r) = 4\pi \int_0^r s^2 \rho(s)\, ds$$

図 3.1 3 次元の球殻を断面で示した図。色の付いた殻の厚さ dr を強調しており、殻の体積は $4\pi r^2 dr$ です。内側の薄灰色の領域が $M(<r)$ に対応します。殻の外にも密度は続き、この円を Plummer 球の外縁とはみなしません。

よくある失敗：軌道が平面内でも、球殻の体積は変わりません

式(3.1)に必要なのは $4\pi r^2$ です。図が 2 次元だからといって $2\pi r$ へ替えません。後者は、面密度から円環内の質量を求める場合の因子です。第 4 章の円盤では、体積密度 ρ とは異なる面密度 Σ を使います。

3.2.2 内側の質量から重力加速度を求める

球対称な殻が内部へ及ぼす重力は、各方向からの寄与が打ち消し合います。一方、殻の外では、その質量を中心へ集めた場合と同じ重力になります。Newton の球殻に関する定理を使うと、半径 $r > 0$ での加速度は、内側の質量だけで決まります。[2]

$$\boldsymbol{a}(\boldsymbol{r}) = -\frac{GM(<r)}{r^3}\boldsymbol{r}. \tag{3.3}$$

向きは中心向き、大きさは $GM(<r)/r^2$ です。点質量の場合と似ていますが、ここでは分子の $M(<r)$ が半径に応じて変わります。

円運動をする星では、重力加速度と向心加速度が等しくなります。円運動速度 v_c は

$$\frac{v_c^2(r)}{r} = \frac{GM(<r)}{r^2}, \qquad v_c^2(r) = \frac{GM(<r)}{r} = r\frac{d\Phi}{dr} \tag{3.4}$$

です。Φ は単位質量当たりの重力ポテンシャルで、$\boldsymbol{a} = -\nabla\Phi$ と定義します。

もう少し詳しく：内側の質量だけでポテンシャルも書けるでしょうか

一般の球対称分布では、$\Phi(r) = -GM(<r)/r$ とは書けません。外側の殻は、内部の力には寄与しなくても、ポテンシャルには寄与するためです。$\Phi(\infty) = 0$ と選ぶと、$r > 0$ について

$$\Phi(r) = -G\left[\frac{M(<r)}{r} + 4\pi\int_r^\infty r'\rho(r')\,dr'\right]$$

です。詳しい導出は文献 [2] の §2.2.1 に譲ります。本章では、次に示す Plummer ポテンシャルの解析式を使います。

3.3 Plummer モデルを導入する

3.3.1 密度とポテンシャルを対応させる

中心で有限の密度を持ち、外側で滑らかに薄くなる簡単なモデルとして、Plummer 球を選びます。密度とポテンシャルは

$$\rho(r) = \frac{3M}{4\pi a^3}\left(1 + \frac{r^2}{a^2}\right)^{-5/2}, \tag{3.5}$$

$$\Phi(r) = -\frac{GM}{\sqrt{r^2 + a^2}} \tag{3.6}$$

です。これらは同じ質量分布を表す一組の式です。文献 [2] の Plummer スケール長 b を、本書では a と書きます。どちらかの式だけで a を変更すると、異なるモデルを混ぜてしまいます。

中心密度は $\rho(0) = 3M/(4\pi a^3)$ です。全質量を固定して a を大きくすると、中心密度が下がり、質量が広く分布します。外側では $\rho \propto r^{-5}$ となり、Φ は点質量の $-GM/r$ へ近づきます。a は銀河の外縁ではなく、中心側と外側の振る舞いが変わる長さの尺度です。

3.3.2 内側の質量を解析的に求める

数値積分を始める前に、比較に使う解析式を求めます。式(3.5)を式(3.2)へ代入すると、

$$M(<r) = 3Ma^2\int_0^r \frac{r'^2}{(r'^2 + a^2)^{5/2}}\,dr' \tag{3.7}$$

です。ここで

$$\frac{d}{dr'}\left[\frac{r'^3}{(r'^2 + a^2)^{3/2}}\right] = \frac{3a^2 r'^2}{(r'^2 + a^2)^{5/2}} \tag{3.8}$$

を使うと、積分の結果は

$$\boxed{M(<r) = M\frac{r^3}{(r^2 + a^2)^{3/2}}} \tag{3.9}$$

になります。この式は、採用した密度から本章で直接計算したものです。$r = 0$ で 0、$r \to \infty$ で M になり、$r > 0$ では単調に増えます。$r = a$ では $M(<a)/M = 2^{-3/2} \simeq 0.3536$ なので、a は半質量半径でもありません。

3.3.3 単位と標準パラメータを決める

前章と同じく、内部単位は kpc、Gyr、$M_\odot$ です。速度は内部では kpc Gyr^{-1} を使い、図や数値の表示でだけ km s^{-1} へ換算します。以下では NumPy を `np`、Matplotlib の `pyplot` を `plt` として読み込んでいるものとします。

```
KM_S_TO_KPC_GYR = 1.0227121650537077
G = 4.30091e-6 * KM_S_TO_KPC_GYR**2  # kpc^3 / (Msun Gyr^2)
mass = 1.0e11                        # 球全体の質量 [Msun]
a = 3.0                              # Plummer スケール長 [kpc]
mu = G * mass
r_tab = 5.0 * a                       # 質量表の範囲 [kpc]。球の外縁ではありません。
n_mass = 256                         # 台形則の区間数。分点数は n_mass + 1 です。
root_xtol = 1.0e-8                    # 二分法の半区間幅の許容値 [kpc]
r0 = 2.0 * a                         # 軌道の初期半径 [kpc]
```

本章の $M = 10^{11} M_\odot$、$a = 3\,\mathrm{kpc}$ は教材用の設定です。特定の実在銀河を再現する値ではありません。$r_{\mathrm{tab}} = 5a = 15\,\mathrm{kpc}$ までの質量表を、標準では 256 区間、257 分点で作ります。`root_xtol` の役割は、二分法の節で説明します。

密度、ポテンシャル、解析的な質量を、それぞれ短い関数にします。`radius` には 1 個の半径も、半径を並べた配列も渡せます。

```
def density_plummer(radius, mass, a):
    """3 次元の質量密度 [Msun/kpc^3]。本文 eq:ch3-density。"""
    radius = np.asarray(radius, dtype=float)
    return 3.0 * mass / (4.0 * np.pi * a**3) * (1.0 + (radius / a)**2)**(-2.5)


def potential_plummer(radius, mass, a):
    """ 単位質量当たりのポテンシャル [kpc^2/Gyr^2]。"""
    return -G * mass / np.sqrt(np.asarray(radius)**2 + a**2)


def mass_plummer_exact(radius, mass, a):
    """ 半径 radius 以内の解析的な質量。本文 eq:ch3-mass-exact。"""
    radius = np.asarray(radius, dtype=float)
    return mass * radius**3 / (radius**2 + a**2)**1.5
```

ここで半径の配列を使うのは、**同じ銀河の半径方向の表**を作るためです。複数の星の運動を同時に計算しているわけではありません。

3.4 台形則で内側の質量を求める

3.4.1 積分する関数を決める

式(3.9)を使えば質量はすぐ求まります。それでも数値積分を行うのは、正解の分かる問題で算法と実装を確かめるためです。解析的に積分できない密度や、分点上の値だけが与えられる場合にも、同じ考え方を使えます。

積分する関数を

$$g(r) = 4\pi r^2 \rho(r), \qquad M(<r) = \int_0^r g(r')\,dr' \tag{3.10}$$

と置きます。g の単位は $M_\odot\,\mathrm{kpc}^{-1}$ で、半径方向に積分すると質量になります。密度 ρ だけを dr で積分するのではありません。

第 1・2 章の時間積分は、未知の位置や速度を次の時刻へ更新する計算でした。今回は、与えられた関数の**定積分**を近似します。どちらも積分ですが、積分する変数と未知量を区別してください。

3.4.2 一つの小区間を台形で近似する

$0 = r_0 < r_1 < \cdots < r_{N_r} = r_{\mathrm{tab}}$ という分点を用意し、$g_j = g(r_j)$ とします。この節の下付き j は半径の分点番号です。軌道の粒子番号や時刻番号ではありません。

小区間 $[r_j, r_{j+1}]$ では、両端の値を結ぶ直線で関数を近似します。幅を $\Delta r_j = r_{j+1} - r_j$ とすると、その直線は

$$g(r) \simeq g_j + \frac{g_{j+1} - g_j}{\Delta r_j}(r - r_j)$$

です。これを小区間で積分すれば、

$$\int_{r_j}^{r_{j+1}} g(r)\,dr \simeq \frac{g_j + g_{j+1}}{2}\Delta r_j \tag{3.11}$$

となります。両端の平均の高さに幅を掛ける、台形の面積です。これが台形則の基本です。[3]

図3.2の注記から、一つの台形の面積を式(3.11)へ対応させてください。

3.4.3 各半径までの質量を順に足す

すべての台形を最後に足すだけでなく、途中の和も保存します。数値積分で得た質量を $\widehat{M}_j$ と書くと、

$$\widehat{M}_0 = 0, \qquad \widehat{M}_{j+1} = \widehat{M}_j + \frac{g_j + g_{j+1}}{2}(r_{j+1} - r_j) \tag{3.12}$$

です。最後の値だけでなく、各分点に対応する累積質量の表が得られます。ハットはここでは数値近似を示すもので、無次元量を意味しません。

まずループで、式をそのまま実装します。

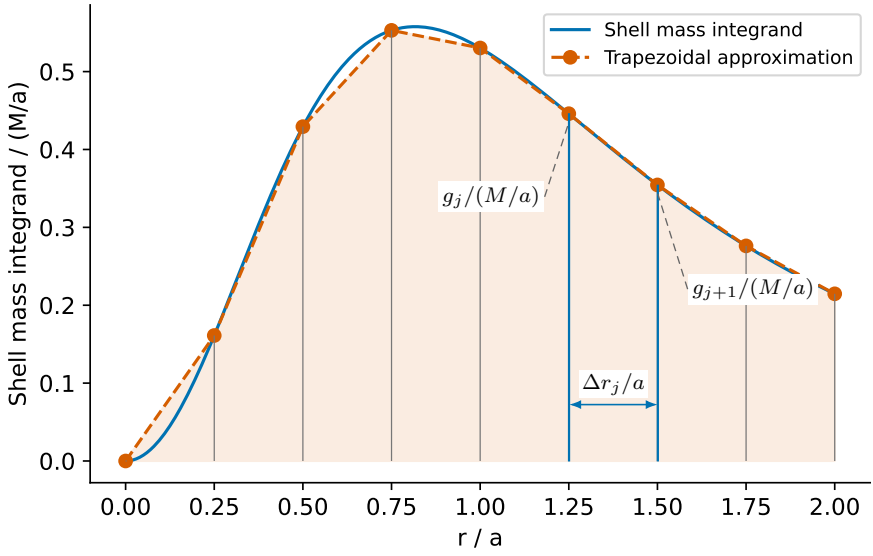


図 3.2　台形則で近似する球殻の質量の被積分関数。説明用に $0 \le r \le 2a$ を 8 区間へ分けています。縦軸は $g/(M/a) = 3(r/a)^2[1+(r/a)^2]^{-5/2}$ です。標準の 256 区間の計算とは分割数が異なります。一つの台形の高さ g_j, g_{j+1} と幅 Δr_j を注記しています。図の軸に合わせ、高さを M/a、幅を a で規格化しています。

```
def cumulative_trapezoid(x, values):
    """ 各分点までの累積積分。本文 eq:ch3-cumulative-trapezoid。"""
    x, values = check_grid(x, values)
    integral = np.zeros_like(x)
    for j in range(len(x) - 1):
        area = 0.5 * (values[j] + values[j + 1]) * (x[j + 1] - x[j])
        integral[j + 1] = integral[j] + area
    return integral
```

`check_grid` は、分点と関数値が同じ長さの 1 次元配列で、値が有限かつ分点が狭義単調増加していることを確認する補助関数です。実装は Notebook に置きます。積分の本体は、台形の面積を計算し、前の和へ足す 2 行です。

この関数へ、半径の表と g の値を渡します。

```
r_grid = np.linspace(0.0, r_tab, n_mass + 1)
rho_grid = density_plummer(r_grid, mass, a)
grid_shell = 4.0 * np.pi * r_grid**2 * rho_grid
mass_numeric = cumulative_trapezoid(r_grid, grid_shell)
mass_exact = mass_plummer_exact(r_grid, mass, a)
```

`mass_numeric[j]` が r_j 以内の質量です。`np.linspace` の最後の引数は分点数なので、区間数に 1 を加えます。

Notebook には、隣接する値をスライスで取り出し、`np.cumsum` で累積和を作る書き方も示します。計算している台形はループ版と同じです。まずは式と各配列要素の対応を理解し、コードが短いことだけを目標にしないようにします。

3.4.4 2 次精度の意味を確認する

滑らかな関数について、一つの幅 Δr の小区間を台形で近似した誤差は $O(\Delta r^3)$ です。固定した範囲を等分すると、小区間の個数は Δr^{-1} に比例します。そのため、小区間の誤差を足した合成台形則の誤差は、通常 $O(\Delta r^2)$ になります。[3]

分割数を 2 倍にして幅を半分にすると、誤差がおよそ $1/4$ になることを期待します。ただし、これは関数が滑らかで刻みが十分小さく、丸め誤差が支配しない範囲での話です。多項式を何次まで厳密に積分できるかという「代数的精度」とは、別の数え方です。本章では、**幅に対する誤差の変化**を確認します。

もう少し詳しく：ほかの数値積分法

Simpson 則は、小区間の関数を放物線で近似して積分する方法です。滑らかな問題では台形則より高い次数を得られます。Gauss 求積のように、分点と重みの選び方を工夫する方法もあります。本章では、表から累積積分を作りやすい台形則を使います。ほかの方法の導出・実装は数値計算の教科書に譲ります。台形則と Simpson 則の比較は、文献 [3] の第 3 章を参照してください。

3.5 数値積分を解析式と比べる

3.5.1 原点でも使える誤差を定義する

数値積分と解析式を同じ分点で比較します。原点では $M(<0)=0$ なので、各半径の質量で割った相対誤差は使いません。球全体の質量 M で規格化し、

$$e_M = \max_{0\le j\le N_r} \frac{|\widehat{M}_j - M(<r_j)|}{M} \tag{3.13}$$

とします。これなら原点を含む表全体を、一つの基準で評価できます。

標準条件の最大誤差は $e_M = 3.67\times10^{-5}$ でした。図3.3がほぼ重なっても、この差は 0 ではありません。図の見た目とは別に、数値で確認することが必要です。

3.5.2 積分範囲を固定して分割する

ここでは $r_{\mathrm{tab}} = 5a$ を変えず、区間数 N_r だけを増やします。

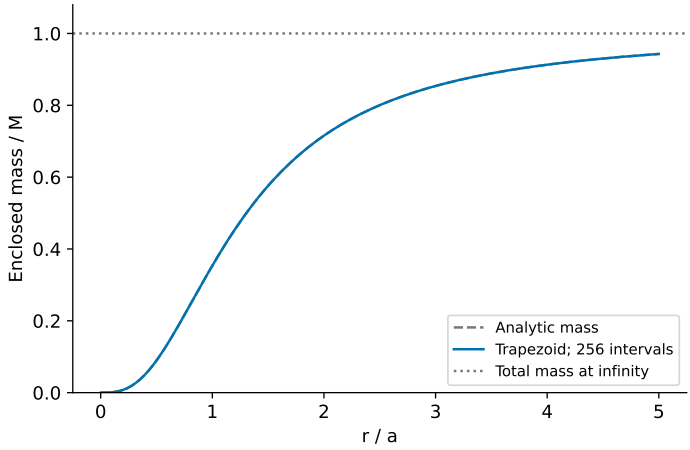


図 3.3　Plummer 球の累積質量。灰色破線が解析式、青実線が $0 \le r \le 5a$ を 256 区間で積分した結果です。両者はこの縮尺ではほぼ重なります。上の点線は球全体の質量であり、$5a$ の内側だけでは到達しません。

```
panel_counts = [20, 40, 80, 160, 320]
mass_rows = []
for count in panel_counts:
    r = np.linspace(0.0, r_tab, count + 1)
    shell = 4.0 * np.pi * r**2 * density_plummer(r, mass, a)
    calculated = cumulative_trapezoid(r, shell)
    expected = mass_plummer_exact(r, mass, a)
    error = np.max(np.abs(calculated - expected)) / mass
    mass_rows.append([count, r_tab / count, error])
mass_convergence = np.asarray(mass_rows)
mass_orders = np.log2(mass_convergence[:-1, 2] / mass_convergence[1:, 2])
```

表 3.1　台形則の分割数試験。積分範囲は全行で $[0, 5a]$ です。p_M は前の行との誤差比から求めた推定次数で、最初の行には比較相手がありません。

N_r	Δr [kpc]	e_M	p_M
20	0.750000	5.87×10^{-3}	—
40	0.375000	1.46×10^{-3}	2.0037
80	0.187500	3.76×10^{-4}	1.9602
160	0.093750	9.40×10^{-5}	2.0014
320	0.046875	2.35×10^{-5}	2.0004

推定次数は、第 1・2 章の時間刻みの試験と同様に、

$$p_M = \frac{\log[e_M(\Delta r)/e_M(\Delta r/2)]}{\log 2} \tag{3.14}$$

で求めます。表3.1の最も細かい側では $p_M = 2.0004$ となり、2 次の収束を確認できます。最大誤差を取る分点の位置も分割数とともに変わるため、粗い側の推定次数が滑らかに 2 へ近づくとは限りません。

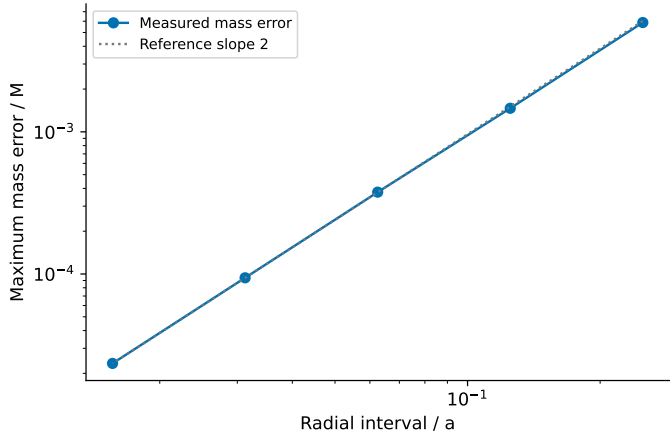


図 3.4　半径方向の幅と質量誤差。点線は、最も細かい分割の実測値を通る傾き 2 の参照線です。実測点を当てはめて傾きを決めた線ではありません。

3.5.3 積分範囲の効果と積分誤差を区別する

同じ $[0, 5a]$ をいくら細かく分けても、得られるのは $M(<5a)$ です。全質量 M にはなりません。式(3.9)から求めた、いくつかの範囲内の質量を表3.2に示します。

表 3.2　有限半径の内側と外側の質量。すべて解析式の値です。範囲外にある質量と、有限刻みによる数値積分誤差を区別します。

r_{tab}/a	$M(<r_{\mathrm{tab}})/M$	範囲外の質量 $/M$
5	0.942866034	0.057133966
10	0.985185337	0.014814663
20	0.996261685	0.003738315

例えば $5a$ より外には、全質量の約 5.7% が残ります。これは台形則の失敗ではありません。全質量まで近づけたいときには積分範囲も広げます。ただし、区間数を固定したまま範囲だけ広げると、中心付近の分割が粗くなります。**範囲を広げることと、同じ範囲を細かく分けることは、別の操作です。**

よくある失敗：最後の値を無条件に全質量へそろえない

`mass_numeric[-1]` を M に合わせて表全体を再規格化すると、積分範囲の外の質量を内側へ配り直してしまいます。本章の M は無限遠まで含めた全質量

です。有限半径で切断した別のモデルを作るときは、その定義を改めて示す必要があります。

3.6 二分法で半質量半径を求める

3.6.1 半径を未知数にする

ここまでは半径を指定して、その内側の質量を求めました。次は、質量を指定して半径を求めます。**半質量半径** r_h は、球全体の質量の半分が入る半径です。

$$M(<r_h) = \frac{M}{2}. \tag{3.15}$$

Plummer モデルでは、解析式を使って比較値も求められます。式(3.9)を M で割って $1/2$ と置き、両辺を $2/3$ 乗すると、

$$\frac{r_h^2}{r_h^2 + a^2} = 2^{-2/3}.$$

これを r_h について整理すると、

$$r_h = \frac{a}{\sqrt{2^{2/3} - 1}} \simeq 1.304766\, a \tag{3.16}$$

です。本章の $a = 3\,\mathrm{kpc}$ では $r_h = 3.914298080\,\mathrm{kpc}$ です。今回も、この解析式だけで答えを出すのではなく、数値的な求め方を検証します。

3.6.2 積分と求根を分ける

二分法は、関数の値が 0 になる場所を探す**求根法**です。本章で計算する関数を

$$F(r) = \frac{\widehat{M}(<r)}{M} - \frac{1}{2} \tag{3.17}$$

とします。$\widehat{M}$ は台形則の結果です。$F < 0$ なら内側の質量が半分より少なく、$F > 0$ なら多いことを意味します。

先ほどの質量表は決まった分点の値だけを持っていますが、二分法ではその間の半径も試します。そこで本章では、**半径を試すたびに $[0, r]$ を同じ区間数で積分し直す**実装にします。表の補間をまだ必要とせず、二分法が呼ぶ関数の意味を直接追えます。

```
def enclosed_mass_trapezoid(radius, mass, a, n_panels):
    """[0, radius] を n_panels 区間で積分します。"""
    check_panels(n_panels)
    if not np.isfinite(radius) or radius < 0.0:
        raise ValueError("radius must be finite and nonnegative.")
    if radius == 0.0:
        return 0.0
    r = np.linspace(0.0, radius, n_panels + 1)
    shell = 4.0 * np.pi * r**2 * density_plummer(r, mass, a)
    return cumulative_trapezoid(r, shell)[-1]


def half_mass_residual(radius, n_panels=n_mass):
    return enclosed_mass_trapezoid(radius, mass, a, n_panels) / mass - 0.5
```

`enclosed_mass_trapezoid` は積分、`half_mass_residual` は求根する方程式を担当します。$r = 0$ なら積分値は 0 として返します。$r > 0$ では、区間幅は r/N_r です。求根の途中では r が変わるので、分割幅も変わります。

3.6.3 符号の変わる区間を半分にする

連続な関数 F について、端点 $r_L < r_U$ で符号が異なれば、その間に少なくとも一つの根があります。Plummer 球の真の累積質量は単調増加するので、半質量半径は一つです。標準の数値積分でも、$[0, 5a]$ の両端で $F(0) < 0$、$F(5a) > 0$ になります。

まず中点を計算します。

$$r_C = \frac{r_L + r_U}{2}. \tag{3.18}$$

$F(r_C)$ が $F(r_L)$ と同符号なら、根は右半分にあるので $r_L \leftarrow r_C$ とします。異符号なら左半分を残し、$r_U \leftarrow r_C$ とします。これを繰り返すのが二分法です。導関数は必要ありません。[3]

根が残る側を選んでいるので、区間の幅は毎回半分になります。ただし、符号が変わるだけで十分なのは、関数がその区間で連続な場合です。不連続な発散点を挟んで符号が違っても、そこに根があるとは限りません。

図3.5で、端点と中点の符号から、次に残す半区間をたどってください。太い線の範囲が次の行の探索区間になります。

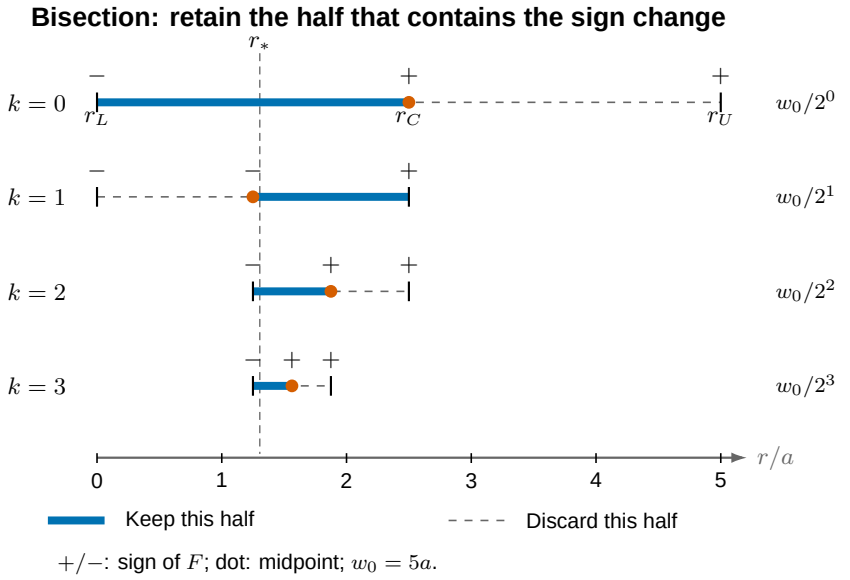


図 3.5　初期区間 $[0, 5a]$ からの二分法の例。点が中点、$+, -$ が評価した F の符号、太い線が次に残す半区間です。k はそれまでの半減回数です。破線 r_* は評価している数値関数の根であり、元の密度の真の半質量半径と必ずしも一致しません。

3.6.4 二分法の終了条件を決める

初期区間の幅を w_0 とします。k 回区間を半分にした後の幅は $w_k = w_0/2^k$ です。その区間の中点を根の近似値とすれば、区間が正しい根を含む限り、中点から根までの距離は半区間幅以下です。

$$|r_C - r_*| \le \frac{w_k}{2} = \frac{w_0}{2^{k+1}}. \tag{3.19}$$

ここで r_* は、**実際に評価している関数 F の根**です。まだ、元の連続的な密度の真の半質量半径 r_h と同じとは限りません。

本章では、半区間幅が許容値 τ_r 以下になったら終了します。τ_r は半径と同じ単位を持ちます。例えば `root_xtol = 1e-8` は 10^{-8} kpc です。無次元の残差 $|F|$ の許容値ではありません。

符号を保つ更新をコードにします。

```
def bisection(func, lower, upper, xtol, max_iter=100):
    """ 半区間幅が xtol 以下になるまで二分します。本文 eq:ch3-bisection。"""
    f_lower, f_upper = check_bisection(func, lower, upper, xtol, max_iter)
    if f_lower == 0.0 or f_upper == 0.0:
        endpoint = lower if f_lower == 0.0 else upper
        return endpoint, np.array([[0, endpoint, endpoint, endpoint, 0.0, 0.0]])
    trace = []
    for k in range(max_iter):
        half_width = 0.5 * (upper - lower)
        midpoint = lower + half_width
        f_mid = finite_value(func, midpoint)
        trace.append([k, lower, upper, midpoint, f_mid, half_width])
        if f_mid == 0.0 or half_width <= xtol:
            return midpoint, np.asarray(trace)
        if midpoint == lower or midpoint == upper:
            raise RuntimeError("The bracket cannot be reduced in floating-point
            ↪  arithmetic.")
        if np.signbit(f_mid) == np.signbit(f_lower):
            lower, f_lower = midpoint, f_mid
        else:
            upper, f_upper = midpoint, f_mid
    raise RuntimeError("Bisection did not reach xtol within max_iter.")
```

`check_bisection` は、区間、許容幅、反復回数、端点の符号を確認します。`finite_value` は、関数値が有限であることを確認します。両者の完全なコードは Notebook に含めます。`np.signbit` は負の符号を持つかどうかを返し、関数値どうしの積を作らずに同符号かを判定できます。

`trace` には、反復番号、左右の端点、中点、残差、半区間幅を保存します。関数値が計算上ちょうど 0 になった場合も終了します。通常の浮動小数点演算を使うため、これは丸めを含めて厳密に根を保証する区間演算の実装ではありません。更新が丸めの限界に達したときや反復上限を超えたときには、未収束として停止します。

3.6.5 実際に半質量半径を求める

```
r_half, root_trace = bisection(half_mass_residual, 0.0, 5.0 * a, root_xtol)
r_half_exact = a / np.sqrt(2.0**(2.0 / 3.0) - 1.0)
print(f"Numerical half-mass radius = {r_half:.9f} kpc")
print(f"Analytic half-mass radius  = {r_half_exact:.9f} kpc")
print(f"Residual of numerical F = {half_mass_residual(r_half):.6e}")
print(f"Residual using exact M = {mass_plummer_exact(r_half, mass, a) / mass - 0.5:.6e}")
```

標準の 256 区間では、数値結果は $\widehat{r}_h = 3.914303805\,\mathrm{kpc}$ でした。区間を半分にした回数は 30 回、中点を評価した回数は 31 回です。最初の区間の中点も評価しているため、二つの回数は 1 だけ異なります。終了時の半区間幅は $6.98 \times 10^{-9}\,\mathrm{kpc}$ です。

もう少し詳しく：ほかの求根法

Newton 法は関数値と導関数から次の近似値を決め、適切な条件では二分法より速く収束しますが、初期値や関数の性質に注意が必要です。割線法は導関数の代わりに二つの点から傾きを見積もります。Brent 法は区間を保つ操作と補間を組み合わせる方法です。本章では、根を挟む区間を追いやすい二分法に集中します。二分法、Newton 法、割線法は文献 [3] の第 9 章を参照してください。Brent 法など、ここで扱わない方法の詳細は数値計算の専門書に譲ります。

3.7 求根の残差と半径の誤差を区別する

3.7.1 二つの誤差を区別する

二分法を終えたときの $|F(\widehat{r}_h)|$ は 3.93×10^{-10} でした。一方、同じ半径を解析的な質量式へ代入したときの残差は、

$$\left| \frac{M(<\widehat{r}_h)}{M} - \frac{1}{2} \right| = 8.12 \times 10^{-7}$$

です。二つの値は一致しません。二分法は台形則で定義した F を 0 へ近づけていますが、その F 自体に積分の誤差が含まれるためです。

実際、真の半径との差は $|\widehat{r}_h - r_h| = 5.73 \times 10^{-6}$ kpc で、二分法の最終半区間幅より大きくなっています。式(3.19)が誤っているのではありません。そこに現れる r_* が、台形則で作った関数の根だからです。

3.7.2 まず積分の分割数を変える

求根の初期区間と許容幅を固定し、質量積分の区間数だけを変えます。

```
root_rows = []
for count in [32, 64, 128, 256, 512]:
    def residual(radius):
        return half_mass_residual(radius, n_panels=count)
    root, trace = bisection(residual, 0.0, 5.0 * a, root_xtol)
    actual_error = abs(root - r_half_exact)
    root_rows.append([count, root, actual_error, abs(residual(root)), trace[-1, 5]])
root_convergence = np.asarray(root_rows)
root_orders = np.log2(root_convergence[:-1, 2] / root_convergence[1:, 2])
```

表3.3では、積分を細かくすると真の半径との差も小さくなります。本例では、その差はおよそ区間数の 2 乗に反比例して減っています。求根の終了条件を厳しくする前に、その関数値をどの精度で計算しているかを見る必要があります。

表 3.3　積分の区間数による半質量半径の変化。二分法の許容半区間幅は全行で 10^{-8} kpc です。小さな数値残差が得られていても、真の半径との差には区間数への依存が残ります。

N_r	$\widehat{r}_h$ [kpc]	$\lvert\widehat{r}_h - r_h\rvert$ [kpc]	$\lvert F(\widehat{r}_h)\rvert$
32	3.914664479	3.66×10^{-4}	7.95×10^{-10}
64	3.914389650	9.16×10^{-5}	5.68×10^{-10}
128	3.914320974	2.29×10^{-5}	3.60×10^{-10}
256	3.914303805	5.73×10^{-6}	3.93×10^{-10}
512	3.914299516	1.44×10^{-6}	8.84×10^{-10}

3.7.3 次に求根の許容幅だけを変える

今度は積分を 64 区間に固定し、τ_r だけを変えます。

表 3.4　積分を 64 区間に固定した許容幅の試験。求根を細かく続けても、積分から来る半径のずれは取り除けません。

τ_r [kpc]	最終半区間幅 [kpc]	真の半径との差 [kpc]	$\lvert F\rvert$
1.00×10^{-2}	7.32×10^{-3}	4.16×10^{-3}	5.76×10^{-4}
1.00×10^{-4}	5.72×10^{-5}	9.63×10^{-5}	6.70×10^{-7}
1.00×10^{-6}	8.94×10^{-7}	9.18×10^{-5}	3.61×10^{-8}
1.00×10^{-8}	6.98×10^{-9}	9.16×10^{-5}	5.68×10^{-10}

許容幅が十分小さくなると、真の半径との差は約 9.16×10^{-5} kpc の付近でほぼ変わらなくなります。これは計算機の丸め誤差まで到達したからではなく、この実験では質量積分の誤差が残っているからです。

よくある失敗：求根の桁数だけで結果の精度を判断しない

数値的な関数の残差が小さくても、もとの方程式を同じ精度で解けたとは限りません。積分の分割数と求根の許容幅を別々に変更して確認します。また、$M/2$ を $\widehat{M}(< r_{\mathrm{tab}})/2$ へ替えると、有限の範囲内にある質量の半分を求める、別の問題になります。

3.8 Plummer 球の重力で星を動かす

3.8.1 加速度は解析式から与える

ここからは、星を表す 1 粒子の時間発展を計算します。質量積分と求根の誤差を軌道へ混ぜないため、**本章の標準軌道計算には解析的な Plummer 加速度を使います**。数値的な質量表は、ここまでの積分・求根の学習と検証に使ったものです。

式(3.6)を微分すると、

$$\frac{d\Phi}{dr} = \frac{GMr}{(r^2 + a^2)^{3/2}}$$

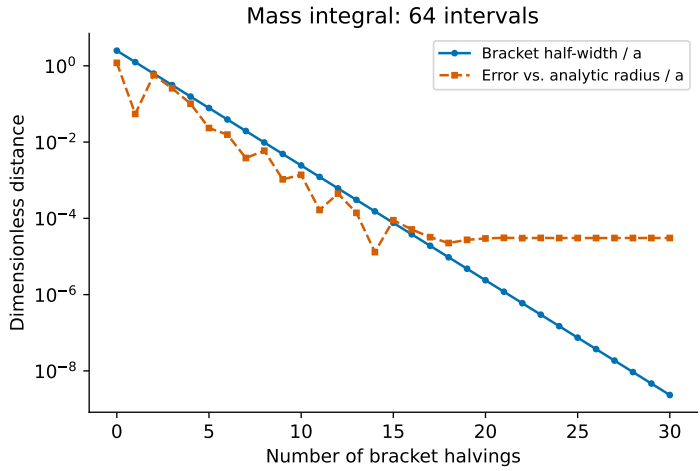


図 3.6　64 区間の質量積分を使った二分法の履歴。青は半区間幅、朱色は中点と解析的な半質量半径の差です。半区間幅は半減を続けますが、真の半径との差は積分誤差の大きさ付近に残ります。どちらも a で規格化しています。

なので、加速度は

$$\boxed{\boldsymbol{a}(\boldsymbol{r}) = -\frac{GM\boldsymbol{r}}{(r^2+a^2)^{3/2}}} \tag{3.20}$$

です。これは式(3.3)へ解析的な $M(<r)$ を代入しても得られます。二つの求め方が一致することが、密度・質量・ポテンシャルの整合性の確認になります。

原点では $\boldsymbol{a}=0$ です。$r \ll a$ では

$$\boldsymbol{a} \simeq -\frac{GM}{a^3}\boldsymbol{r}, \tag{3.21}$$

となり、位置に比例する調和振動子の形になります。一方、$r \gg a$ では点質量の重力へ近づきます。中心で加速度が発散しないのは、質量分布を広げたモデルの性質であり、計算エラーを隠す処置ではありません。

3.8.2 円運動速度も同じモデルから求める

式(3.4)に代入すると、

$$v_c^2(r) = \frac{GMr^2}{(r^2+a^2)^{3/2}}, \qquad \Omega_c(r) = \frac{v_c(r)}{r} \tag{3.22}$$

です。$r>0$ の円軌道周期は $T_c(r) = 2\pi/\Omega_c(r)$ になります。中心近くでは $v_c \propto r$、十分外では $v_c \propto r^{-1/2}$ です。[2]

まずは成分で加速度を計算します。

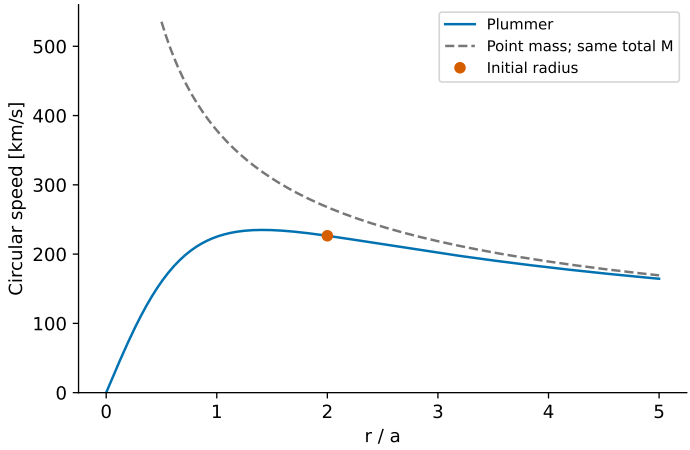


図 3.7　同じ全質量を持つ Plummer 球と点質量の円運動速度。丸は本章の初期半径 $r_0 = 2a$ です。点質量の曲線は $r \geq 0.5a$ だけを示しています。原点に向かって発散する点質量の速度と、中心で 0 へ向かう Plummer 球の速度は異なります。

```
x, y = r0, 0.0
r2 = x**2 + y**2
ax = -G * mass * x / (r2 + a**2)**1.5
ay = -G * mass * y / (r2 + a**2)**1.5
print(f"ax = {ax:.6f}, ay = {ay:.6f} [kpc/Gyr^2]")
```

本章の初期位置では $\boldsymbol{a} \simeq (-8.9413 \times 10^3, 0)\,\mathrm{kpc\,Gyr^{-2}}$ です。この計算を短い関数にします。

```
def acceleration_plummer(pos, mass, a):
    """ 原点に固定した Plummer 球。1 粒子の位置・加速度は (1, 2) です。"""
    x, y = pos[0]
    r2 = x**2 + y**2
    ax = -G * mass * x / (r2 + a**2)**1.5
    ay = -G * mass * y / (r2 + a**2)**1.5
    return np.array([[ax, ay]])


def circular_speed_plummer(radius, mass, a):
    radius = np.asarray(radius, dtype=float)
    return np.sqrt(G * mass * radius**2 / (radius**2 + a**2)**1.5)
```

`pos` と返り値は、1 粒子なので形が `(1, 2)` です。多数の粒子への一般化は、第 4 章で行います。円運動速度の関数では、半径ごとの表を描けるよう配列も受け取ります。

よくある失敗：点質量の円運動速度を流用しない

Plummer 球の内部では、全質量 M を使った $\sqrt{GM/r}$ は円運動速度ではありません。力のモデルを変えたら、初期速度も同じモデルから計算し直します。a を変えて円軌道を比較するときも同様です。

3.9 KDK で円軌道と非円軌道を計算する

3.9.1 変える初期値を一つにする

初期半径を $r_0 = 2a = 6\,\mathrm{kpc}$ とし、初期位置と速度を

$$\boldsymbol{r}(0) = (r_0, 0), \qquad \boldsymbol{v}(0) = (0, \eta v_c(r_0)) \tag{3.23}$$

と置きます。$\eta = 1$ が円軌道、$\eta = 0.7$ が比較する非円軌道です。どちらも動径方向の初期速度は 0 で、反時計回りに動き始めます。

$0 < \eta < 1$ では、円運動を保つために必要な接線速度より遅いため、初期半径から内側へ動き始めます。この初期位置は遠点です。ただし、第 2 章の Kepler 問題とは違い、その後の軌道は一般には閉じた楕円になりません。**非円軌道を最初から「楕円」と呼ばないようにします。**

```
v_c = circular_speed_plummer(r0, mass, a)
t_c = 2.0 * np.pi * r0 / v_c
n_periods = 5
steps_per_period = 1000
n_steps = n_periods * steps_per_period
t_end = n_periods * t_c
dt = t_end / n_steps
eta_values = {"Circular": 1.0, "Noncircular": 0.7}
```

表 3.5 本章の標準軌道条件。二つのケースで共通の基準周期 T_c を使います。これは非円軌道の動径振動の周期を表す記号ではありません。

量	設定または計算値
M, a	$10^{11} M_\odot$, 3 kpc
r_0	$6\,\mathrm{kpc} = 2a$
$v_c(r_0)$	$226.475925\,\mathrm{km\,s^{-1}}$
$T_c = 2\pi r_0 / v_c(r_0)$	$0.162763\,\mathrm{Gyr}$
η	$1, 0.7$
t_{end}	$5T_c = 0.813815\,\mathrm{Gyr}$
N_{step}, Δt	5000, $T_c/1000 = 0.162763\,\mathrm{Myr}$

初期状態を作る関数は、位置と速度を横につないだ `(1, 4)` の配列を返します。

```
def make_initial_state(radius, eta, mass, a):
    check_model(mass, a)
    if not np.isfinite(radius) or radius <= 0.0 or not np.isfinite(eta):
        raise ValueError("Initial radius must be positive and eta finite.")
    pos = np.array([[radius, 0.0]])
    vel = np.array([[0.0, eta * circular_speed_plummer(radius, mass, a)]])
    return np.column_stack((pos, vel))
```

`check_model` は、M と a が有限かつ正であることを確認する関数です。KDK では、まとめた状態から位置と速度を分けて取り出し、更新します。

3.9.2 前章の積分法を再利用する

KDK の計算は、半ステップの速度更新、1 ステップの位置更新、新しい位置での加速度評価、残りの半ステップの速度更新です。

```
def leapfrog_step(acceleration, pos, vel, acc, dt, mass, a):
    """KDK。最後の加速度を次のステップへ渡します。"""
    vel_half = vel + 0.5 * dt * acc
    pos_new = pos + dt * vel_half
    acc_new = acceleration(pos_new, mass, a)
    vel_new = vel_half + 0.5 * dt * acc_new
    return pos_new, vel_new, acc_new
```

前章から変えたのは、重力の関数へ渡すパラメータです。今回は二つの質量の配列ではなく、1 個の銀河の `mass` と `a` を渡します。KDK の 4 行と、最後の加速度を再利用する構造は同じです。

```
def integrate_leapfrog(acceleration, state0, t_end, n_steps, mass, a):
    check_model(mass, a)
    state, times, history, dt = prepare_history(state0, t_end, n_steps)
    pos, vel = state[:, :2].copy(), state[:, 2:].copy()
    acc = acceleration(pos, mass, a)
    for n in range(n_steps):
        pos, vel, acc = leapfrog_step(acceleration, pos, vel, acc, dt, mass, a)
        history[n + 1] = np.column_stack((pos, vel))
        if not np.all(np.isfinite(history[n + 1])):
            raise FloatingPointError("Non-finite state; check the model and time step.")
    return times, history
```

`prepare_history` は、初期値をコピーし、時刻の配列と保存領域を作る補助関数です。前章と同じ役割ですが、今回は初期状態が `(1, 4)` であることを確認します。履歴は `(n_steps + 1, 1, 4)` です。保存する速度は、最後の半ステップの更新を終えた整数時刻の値です。

5000 ステップなら、加速度関数の評価は初回を含めて 5001 回です。台形則の区間数や二分法の反復回数は、この回数に含めません。いま評価している重力には解析式を使っています。

3.9.3 同じ時間まで独立に計算する

```
cases = {}
start = perf_counter()
for name, eta in eta_values.items():
    initial = make_initial_state(r0, eta, mass, a)
    times, history = integrate_leapfrog(acceleration_plummer, initial, t_end, n_steps,
    ↪  mass, a)
    cases[name] = {"eta": eta, "initial": initial, "times": times, "history": history}
```

一つのケースの最終状態を、別のケースの初期状態に流用しません。いずれも式(3.23)から作り直します。実行時間は環境に依存するので、Notebook で `perf_counter` を使って測ります。

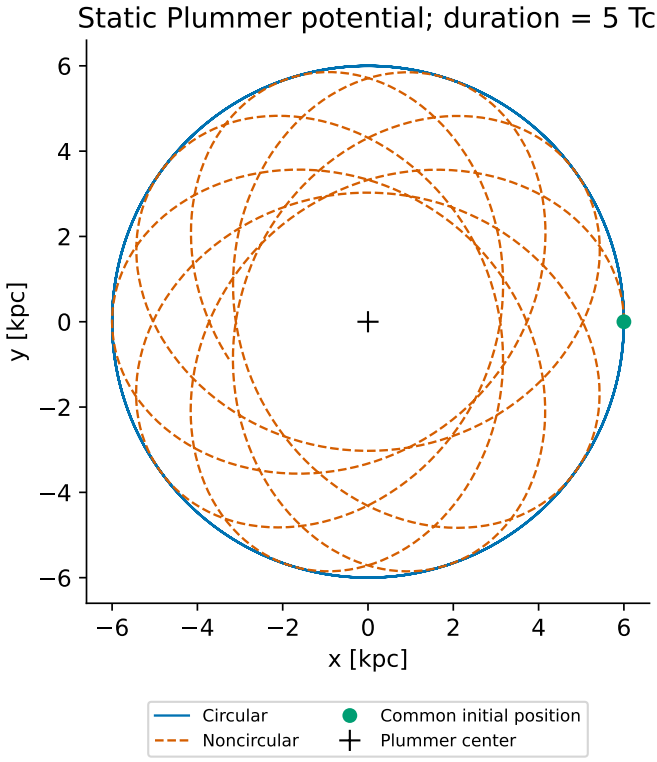


図 3.8 静的な Plummer 球中の円軌道と非円軌道。青実線が $\eta = 1$、朱色破線が $\eta = 0.7$ で、どちらも $5T_c$ まで計算しています。中心の十字はポテンシャルの中心であり、追加した点質量ではありません。丸が共通の初期位置です。

円軌道では半径がほぼ一定ですが、非円軌道では半径が増減し、近点・遠点の方向も変わります。このような軌道はロゼット軌道と呼ばれます。球対称でも、動径方向の振動と方位角方向の回転が、Kepler 問題と同じ周期関係になるとは限りま

せん。[2] 数値誤差による軌道のずれと区別するため、後で時間刻みを細かくした結果も比べます。

Notebook には、保存済みの履歴を使うアニメーションも含めます。画面上の再生間隔は物理的な時間刻みとは別で、動画の速度を変えても数値軌道は変わりません。

3.10 保存量と時間刻みを確認する

3.10.1 確認するエネルギーを決める

固定した銀河ポテンシャル中のテスト粒子なので、確認するのは粒子の**単位質量当たり**のエネルギーと角運動量です。

$$\varepsilon = \frac{1}{2}(v_x^2 + v_y^2) - \frac{GM}{\sqrt{r^2 + a^2}}, \qquad \ell_z = x v_y - y v_x. \tag{3.24}$$

Φ は時間に依存しないので、

$$\frac{d\varepsilon}{dt} = \boldsymbol{v} \cdot \boldsymbol{a} + \nabla\Phi \cdot \boldsymbol{v} = 0$$

です。力は中心向きなので、$d\ell_z/dt = x a_y - y a_x = 0$ でもあります。第 2 章の二つの質量を掛けた全エネルギーや、Plummer 球自身の自己重力エネルギーを計算しているわけではありません。

```
def diagnostics(history, mass, a):
    pos, vel = history[:, 0, :2], history[:, 0, 2:]
    radius = np.sqrt(np.sum(pos**2, axis=1))
    energy = 0.5 * np.sum(vel**2, axis=1) + potential_plummer(radius, mass, a)
    angular = pos[:, 0] * vel[:, 1] - pos[:, 1] * vel[:, 0]
    return radius, energy, angular
```

位置と速度を同じ保存時刻から取り出します。Plummer ポテンシャルを使って運動した結果に、点質量の $-GM/r$ を足してエネルギーを診断しないようにします。

3.10.2 有限刻みによる差を測る

今回の 2 例では $\varepsilon(0)$ と $\ell_z(0)$ がともに 0 ではないので、その絶対値で規格化できます。保存した全時刻での最大値を

$$e_\varepsilon = \max_n \frac{|\varepsilon^n - \varepsilon^0|}{|\varepsilon^0|}, \qquad e_\ell = \max_n \frac{|\ell_z^n - \ell_z^0|}{|\ell_z^0|} \tag{3.25}$$

とします。初期値が 0 の別の軌道では、この規格化は使わず、第 2 章のように正の基準量を選ぶ必要があります。

KDK では中心力による kick も、速度を固定する drift も角運動量を保つため、e_ℓ は丸め誤差程度になります。一方、エネルギーを毎ステップ厳密に保つ方法ではないので、円軌道では $e_\varepsilon = 1.62 \times 10^{-10}$、非円軌道では 1.83×10^{-5} という有限の差が現れました。保存量が小さくずれることと、軌道上の位置が同じ精度で正しいことは、別に調べます。

表 3.6　標準条件の保存量と最小保存半径。両例とも $5T_c$ まで 5000 ステップで計算しています。角運動量の末尾の桁は実行環境で多少変わります。

軌道	e_ε	e_ℓ	保存された最小半径 [kpc]
円	1.62×10^{-10}	1.08×10^{-14}	6.000000
非円	1.83×10^{-5}	9.47×10^{-15}	3.027208

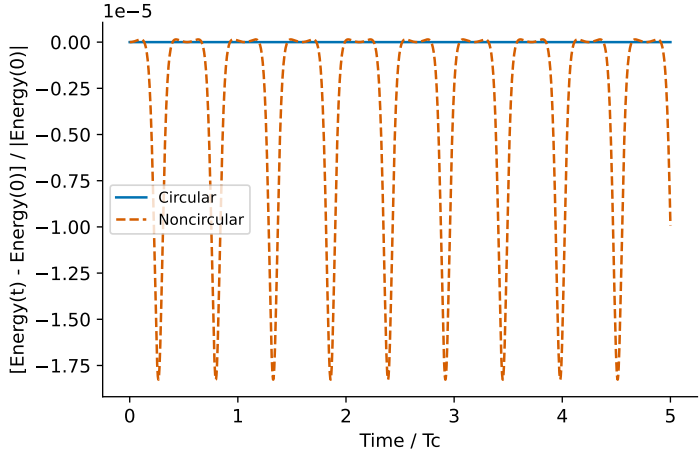


図 3.9　粒子の比エネルギーの初期値からの符号付きの差。円軌道の差は小さく、この縮尺では 0 付近に重なります。非円軌道では半径の増減に伴い差が変動します。これは摩擦によるエネルギー損失を入れた計算ではありません。

3.10.3 円軌道を同じ時刻の解析位置と比べる

円軌道では、$\Omega_c = v_c(r_0)/r_0$ から解析的な位置を

$$\boldsymbol{r}_{\mathrm{exact}}(t) = r_0(\cos \Omega_c t, \sin \Omega_c t) \tag{3.26}$$

と書けます。同じ時刻の数値解との最大の距離を、初期半径で規格化します。

$$e_r = \max_n \frac{|\boldsymbol{r}^n - \boldsymbol{r}_{\mathrm{exact}}(t_n)|}{r_0}. \tag{3.27}$$

半径の差だけではないので、回転の位相がずれた場合も検出できます。

この試験だけ最終時刻を T_c とし、100、200、400、800 ステップで比較します。$5T_c$ の標準図とは計算時間が異なりますが、試験系列の中では同じ時刻まで計算します。

```
def exact_circle(times, radius, mass, a):
    omega = circular_speed_plummer(radius, mass, a) / radius
    phase = omega * np.asarray(times)
    return np.column_stack((radius * np.cos(phase), radius * np.sin(phase)))


orbit_rows = []
initial_circle = cases["Circular"]["initial"]
for count in [100, 200, 400, 800]:
    t, h = integrate_leapfrog(acceleration_plummer, initial_circle, t_c, count, mass, a)
    exact = exact_circle(t, r0, mass, a)
    position_error = np.max(np.linalg.norm(h[:, 0, :2] - exact, axis=1)) / r0
    orbit_rows.append([count, t_c / count, position_error])
orbit_convergence = np.asarray(orbit_rows)
orbit_orders = np.log2(orbit_convergence[:-1, 2] / orbit_convergence[1:, 2])
```

表 3.7　Plummer 球中の円軌道の時間刻み試験。最終時刻は全行で解析的な円軌道周期 T_c です。p_r は前の行との位置誤差の比から求めます。

N_{step}	Δt [Myr]	e_r	p_r
100	1.627630	3.25×10^{-3}	—
200	0.813815	8.12×10^{-4}	2.00052
400	0.406907	2.03×10^{-4}	2.00020
800	0.203454	5.07×10^{-5}	2.00005

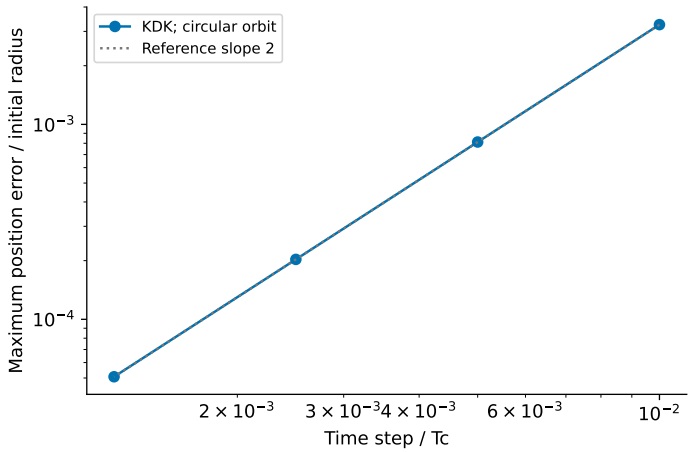


図 3.10　Plummer 球中の円軌道に対する位置誤差。横軸は $\Delta t/T_c$ です。点線は、最も細かい刻みの実測点を通る傾き 2 の参照線です。

最も細かい側の推定次数は $p_r = 2.00005$ で、KDK の 2 次精度を確認できました。重力場を点質量から Plummer 球へ変えても、滑らかな軌道を十分小さい刻みで追

う場合の次数は同じです。円軌道のエネルギー誤差には特殊な打ち消しが生じる場合があるので、その誤差だけから解法の次数を判断しません。

3.10.4 閉じない非円軌道の精度を確かめる

非円軌道では、T_c や $5T_c$ 後に初期位置へ戻るとは限りません。初期位置へ戻らない量を、そのまま誤差にしてはいけません。本章では、同じ $5T_c$ まで異なる刻みで計算した終点を比較します。

刻み h と $h/2$ による終点を使い、

$$d_h = \frac{|\boldsymbol{r}_h(t_{\mathrm{end}}) - \boldsymbol{r}_{h/2}(t_{\mathrm{end}})|}{r_0} \tag{3.28}$$

とします。もし $\boldsymbol{r}_h = \boldsymbol{r}_{\mathrm{true}} + \boldsymbol{C}h^2 + \cdots$ で主要な誤差を表せるなら、差を取ると $(3/4)\boldsymbol{C}h^2$ が残るので、$d_h/d_{h/2}$ は約 4 になります。

表 3.8 非円軌道の終点の自己収束試験。すべて $5T_c$ まで計算します。各行は、隣接する二つの刻みの数値解の差です。真の解との誤差ではありません。

粗い計算のステップ数	細かい計算のステップ数	終点の差 $/r_0$	前の行との比
1250	2500	8.98×10^{-4}	—
2500	5000	2.25×10^{-4}	3.99969
5000	10000	5.61×10^{-5}	3.99992

細かい側の比は 3.99992 で、およそ 4 でした。図3.8のロゼット状の運動が、刻みを細かくすると Kepler 楕円へ変わるわけではありません。ただし、自己収束の確認だけで、力の式そのものが正しいと証明できるわけでもありません。解析的な円軌道、密度と力の関係、保存量と合わせて判断します。

3.11 三つの数値計算の役割を区別する

ここまでの計算では、同じ Plummer モデルに対して、異なる目的の算法を使いました。どの入力を変えると、どの誤差を調べることになるかを整理しておきます。

操作	固定するもの	確認するもの
台形則の区間を細かくする	積分範囲、密度モデル	解析的な質量との差
二分法の許容幅を小さくする	質量積分の区間数、求根区間	数値的な関数の根の精度
KDK の時間刻みを小さくする	初期条件、重力場、最終時刻	軌道位置と保存量の変化

a を変えることは、これらの数値精度を調整することとは違います。密度分布と重力そのものを変更する、**物理モデルの変更**です。例えば M と r_0 を固定して a を大きくすると、内側の質量が減り、円運動速度は小さくなります。円軌道を比較す

るのか、同じ位置・速度から異なるポテンシャルでの応答を比べるのかによって、そろえる初期条件も変わります。

また、本章では軌道に解析的な力を使っているので、台形則の区間数を変えても、そのまま軌道の計算結果が変わることはありません。数値的な質量表から力を計算する実装へ拡張した場合には、空間の積分・補間誤差と時間積分誤差を、改めて分けて確かめる必要があります。そこまでの拡張は本章の必須課題にはしません。

3.12 本章のまとめ

本章では、質量に広がりを持つ静的な銀河を、球対称の Plummer モデルで表しました。密度を球殻の体積で重み付けして積分すると、内側の質量が得られます。球対称の重力加速度と円運動速度は、その内側の質量から決まります。一方、ポテンシャルには外側の質量も寄与するため、力と同じ置き換えを無条件に行いません。

台形則では、小区間の両端を結ぶ直線の下の面積を足しました。積分範囲を固定して幅を半分にすると、質量の誤差がおよそ $1/4$ になることを確認しました。二分法では、半質量半径を挟む区間を保って半分にしました。ただし、二分法の残差を小さくしても、関数値に含まれる質量積分の誤差までは消えません。

最後に、解析的な Plummer 加速度を KDK へ渡して、円軌道と非円軌道を計算しました。固定した球対称場では、粒子の比エネルギーと比角運動量を確認できます。円軌道の解析位置と、非円軌道の刻みを変えた比較も使い、保存量だけに頼らず結果を確かめました。

次の第 4 章では、1 個のテスト粒子から多数の粒子へ進みます。円盤上の分布を作るために、台形則による累積積分を再利用し、線形補間と乱数を組み合わせます。見た目に円盤状の粒子を配置しても、その粒子自身の自己重力を計算するわけではない、という区別を引き継ぎます。

演習

演習 3.1　確認：密度・質量・円運動速度　$M = 10^{11} M_\odot$、$a = 3\,\mathrm{kpc}$ について、中心密度 $\rho(0)$、$M(<a)/M$、半質量半径 r_h を計算してください。また、$r = 2a$ での円運動速度を $\mathrm{km\,s^{-1}}$ と内部単位の両方で示してください。軌道を 2 次元で計算する場合でも、質量積分に $4\pi r^2$ が必要な理由を説明してください。

演習 3.2　確認：台形則の配列　$[0, 5a]$ を $N_r = 40$ 区間に分けたときの分点数と区間幅を求めてください。`cumulative_trapezoid` のループで、`integral[j + 1]` が何を表すか説明してください。密度ではなく $g = 4\pi r^2 \rho$ を渡す理由を、単位からも確かめてください。

演習 3.3　応用：範囲と分割数を分ける　$[0, 5a]$ の区間数を増やす実験と、Δr を固定して範囲を $5a, 10a, 20a$ へ広げる実験を行ってください。数値積分の誤差と範

囲外の質量を別々に示してください。全質量 M と $M(<5a)$ の違いを、台形則の誤差だけで説明できるでしょうか。

演習 3.4　確認：二分法の反復回数　初期区間を $[0, 15]\,\mathrm{kpc}$ としたとき、半区間幅を $10^{-8}\,\mathrm{kpc}$ 以下にするには何回の半減が必要か求めてください。半減回数と中点評価回数の違いも示してください。また、$|F(\widehat{r}_h)|$ が小さくても、$|\widehat{r}_h - r_h|$ が同じ程度には小さくならない理由を説明してください。

演習 3.5　応用：球全体の半分と、範囲内の半分　求根式の右辺を $M/2$ から $M(<5a)/2$ へ変えて、二つの半径を比較してください。先に解析式で違いを確認し、その後、数値積分で作る関数を二分法で解いてください。どちらが小さくなるでしょうか。この変更が単なる数値精度の改善ではない理由を述べてください。

演習 3.6　応用：スケール長を変える　M と $r_0 = 6\,\mathrm{kpc}$ を固定し、$a = 1.5, 3, 6\,\mathrm{kpc}$ の円運動速度と半質量半径を求めてください。それぞれに対応する円運動速度で初期条件を作り、円軌道を計算してください。すべてに同じ初期速度を与えた場合とは、何を比較している点が違うでしょうか。

演習 3.7　発展：非円軌道の近点を求根で調べる　$\eta = 0.7$ の初期比エネルギー ε_0、比角運動量 $\ell_{z,0}$ を用いて、

$$\Phi(r) + \frac{\ell_{z,0}^2}{2r^2} - \varepsilon_0 = 0 \tag{3.29}$$

の根を二分法で求めてください。この式は、動径速度が 0 になる条件です。初期遠点 r_0 ではない内側の根を探すため、区間を $[0.05r_0, 0.95r_0]$ とします。関数の符号を確認してから求根し、軌道の保存された最小半径と比較してください。両者の違いには何が含まれるでしょうか。